26-1 프로젝트 수강신청서 및 계획서 내용

> 이 문서는 `26-1 프로젝트 수강신청서 및 계획서 - 양식.docx` 에 작성할 내용입니다.

1. 기본 정보

항목	내용
학번	C235180
성명	서민지
학과	컴퓨터공학과
지도교수	⚠️ **(교수님 성함 기재 필요)**
연락처	⚠️ **(이메일/전화번호 기재 필요)**

2. 프로젝트 제목

OS 스케줄링 알고리즘을 활용한 멀티테넌트 LLM API 요청 관리 시스템

영문: Multi-tenant LLM API Request Management System using OS Scheduling Algorithms

3. 프로젝트 개요

3.1 연구 배경

ChatGPT, Claude 등 대규모 언어 모델(LLM) API의 사용이 급증하고 있습니다. 2024년 기준 글로벌 LLM API 시장 규모는 전년 대비 2배 이상 성장하였으며(Grand View Research, 2024), 이에 따라 다중 사용자 환경에서 API 요청을 효율적으로 관리해야 할 필요성이 커지고 있습니다. 현재 많은 LLM 서비스는 선착순(FCFS) 처리나 단순

Rate Limiting에 의존하고 있어, 긴 요청이 짧은 요청을 지연시키는 호위 효과(Convoy Effect), 사용자 등급에 따른 차별화 부재, 우선순위 없는 비효율적 처리 등의 문제가 발생합니다.

운영체제 수업에서 프로세스 스케줄링을 학습하며 "실제 시스템에서도 이 알고리즘이 효과적일까?"라는 의문을 갖게 되었고, 이를 LLM API 관리에 적용해 보고자 본 주제를 선정하였습니다.

3.2 해결 방안

운영체제의 CPU 스케줄링 알고리즘을 LLM API 요청 관리에 적용합니다.

알고리즘	설명	OS 수업 연관 개념
FCFS	선착순 처리 (베이스라인)	기본 스케줄링
Priority	긴급 요청 우선 처리 + Aging으로 기아 방지	우선순위 스케줄링, 노화 기법
MLFQ	요청 길이에 따른 동적 큐 이동	다단계 피드백 큐
WFQ	테넌트 가중치 기반 공정 배분	네트워크 스케줄링의 공정 큐잉을 LLM에 적용

3.3 OS 개념 매핑

OS 개념	본 프로젝트 적용
프로세스	LLM API 요청
CPU 시간	API 응답 시간
Ready Queue	요청 대기열
Starvation / Aging	장시간 대기 요청 / 우선순위 상승

4. 목표 및 기대효과

4.1 개발 목표

목표	상세 내용	측정 방법
알고리즘 구현	4가지 스케줄링 알고리즘 구현	단위 테스트 통과
REST API	요청 등록, 조회, 통계 등 핵심 API	curl/Postman 테스트

목표	상세 내용	측정 방법
LLM 연동	Ollama 로컬 LLM 연동	실제 응답 확인
성능 비교	알고리즘별 성능 측정 및 분석	실험 데이터 분석

4.2 기대효과

학습 성과: OS 스케줄링 이론을 실제 시스템에 적용함으로써 이론과 실무의 연결을 경험하고, Node.js/Express 기반 백엔드 개발 및 실험 설계·정량적 분석 역량을 습득할 수 있습니다.

학술적 의의: OS 스케줄링 이론을 AI 시스템에 적용한 사례로서, 4가지 알고리즘의 정량적 성능 비교 데이터와 Jain's Fairness Index를 활용한 공정성 분석 결과를 제시합니다.

실용적 가치: 긴급 요청의 우선 처리로 서비스 응답성을 향상하고, 멀티테넌트 환경에서 가중치 기반 공정한 자원 배분을 구현합니다.

5. 기술 스택

분류	기술	선정 이유
언어/런타임	JavaScript (ES2024) / Node.js 20 LTS	학습 용이, 안정성
웹 프레임워크	Express.js 4.18	간결한 REST API
데이터 저장	JSON 파일	별도 패키지 불필요
테스트	Jest 29.x	표준 프레임워크
LLM	Ollama (로컬)	무료 실행, 외부 API 비용 불필요

학부생 1명이 한 학기 내 구현 가능하도록 의존성을 최소화(3개 패키지)하였으며, OS 알고리즘 이해에 집중할 수 있는 단순한 구조를 택하였습니다.

6. 개발 일정

주차	기간	주요 활동	산출물
1-2	3/1 - 3/14	요구사항 분석, 설계	계획서, 설계 문서
3-6	3/15 - 4/11	4가지 스케줄러 구현	스케줄러 모듈
7-8	4/12 - 4/25	API 개발, 테스트	REST API, 테스트
9-10	4/26 - 5/9	실험, 데이터 수집	성능 비교 데이터
11-13	5/10 - 5/31	보고서 작성, 발표 준비	최종 보고서, PPT

7. 시스템 구조

시스템은 **클라이언트 → REST API → 스케줄러 매니저 → 메모리 큐/JSON 저장소 → Ollama LLM**의 흐름으로 구성됩니다. 이 구조는 OS 커널 위에서 동작하는 사용자 프로세스 모델을 응용한 것입니다.

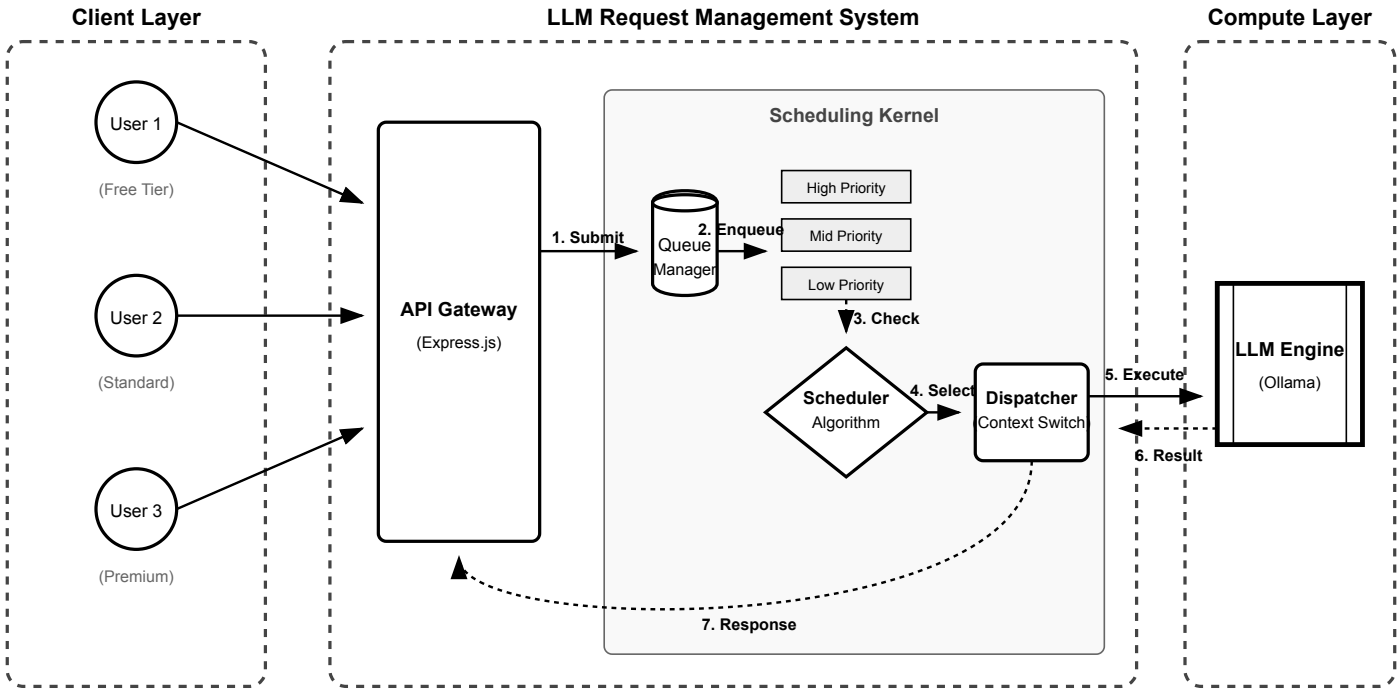

그림 1. 시스템 아키텍처 다이어그램. 클라이언트 요청이 API Gateway를 거쳐 스케줄러에 의해 Ready Queue에 적재되고, 선별된 요청이 LLM 엔진으로 디스패치되는 과정을 보여줍니다.

핵심 모듈과 예상 코드량은 다음과 같습니다.

모듈	설명	예상 코드량
스케줄러 (4종)	FCFS, Priority+Aging, MLFQ, WFQ	~460줄
기타 모듈	큐, 저장소, API, LLM 클라이언트	~540줄
전체 합계		**~1,000줄**

8. 연구 질문 및 예상 성과

본 프로젝트는 목표 수치를 사전에 정하지 않고, 실험 결과를 관찰·분석하는 **탐구적 연구** 방식으로 진행합니다.

연구 질문	알고리즘	핵심 탐구 내용
RQ1	Priority	URGENT 요청은 낮은 우선순위 요청보다 얼마나 빠르게 처리되는가?
RQ2	MLFQ	다양한 길이의 작업이 혼재된 환경에서 어떤 적응성을 보이는가?
RQ3	WFQ	가중치에 비례하는 서비스 차등화를 달성하는가?

예비 테스트를 통해 Priority 스케줄링에서 URGENT 요청의 대기시간이 FCFS 대비 크게 감소하는 가능성을 확인하였으며, 본격적인 실험에서 조건별 결과를 정밀하게 비교할 예정입니다.

품질 목표: 테스트 커버리지 85% 이상, 테스트 통과율 100%, 전체 코드 약 1,000줄

한계점: 본 프로젝트는 로컬 환경에서의 개념 증명(PoC) 수준이며, 대규모 트래픽이나 분산 환경에서의 검증은 향후 과제로 남깁니다.

9. 위험 요소 및 대응

주요 위험으로 Ollama 연동 실패(Mock 객체로 대체), 알고리즘 구현 난이도(OS 수업 자료 참조 및 점진적 구현), 일정 지연(FCFS+Priority 우선 구현으로 최소 결과물 확보)을 식별하였으며, 각각에 대한 대응 방안을 마련하였습니다.

10. AI 활용 및 연구 윤리 준수

본 프로젝트의 핵심 아이디어, 설계, 그리고 알고리즘의 논리적 검증은 제가 주도적으로 수행합니다. 단, 코딩 생산성 향상과 문서 작성의 효율화를 위해 생성형 AI(GitHub Copilot 등)를 보조 도구로 활용할 계획이며, AI가 생성한 모든 결과물은 제가 직접 검토하고 수정한 후 프로젝트에 반영함을 밝힙니다. 본 계획서 작성 과정에서도 아이디어 구체화 및 초안 작성 단계에서 AI의 보조를 받았으며, 최종 내용은 제가 직접 검수 및 수정하였습니다.

작성일: 2026년 2월 8일

작성자: 서민지 (서명)

이 문서의 내용을 docx 양식에 복사하여 사용하세요.